

NAME: CH.SRI LAKSHMI

REGISTER NO: 192311342

COURSE CODE: CSA1511

COURSE NAME: CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSESSMENT TOOL-15

Hadoop Ecosystem and Big Data Constraint-Based Design

1. Design an HDFS-Based Storage Solution for a Media Platform

Introduction

A media platform stores large amounts of videos, images, audio files, metadata, and user-generated content. The storage system must provide high availability while controlling storage costs. HDFS is suitable because it provides distributed storage, fault tolerance, and scalability.

Proposed Design

The proposed HDFS architecture consists of:

- **NameNode:** Maintains the HDFS namespace and metadata.
- **Secondary/Checkpoint Node:** Periodically creates checkpoints of NameNode metadata.
- **DataNodes:** Store actual media blocks across multiple physical machines.
- **Replication Factor:** Set to **3**, which is the maximum allowed under the constraint.
- **Block Size:** Use a large block size such as **256 MB or 512 MB** for large media files.
- **Rack Awareness:** Distribute replicas across different racks to protect against rack-level failures.
- **Data Compression:** Compress metadata, thumbnails, and other compressible files to reduce storage consumption.
- **Hot/Cold Storage Strategy:** Frequently accessed media can remain on high-performance storage, while older content can be moved to lower-cost storage.

Availability Strategy

Each important HDFS block is replicated three times. The replicas are placed on different DataNodes and, where possible, different racks. If one DataNode fails, another replica can immediately serve the data.

The NameNode should also be configured with **High Availability using Active and Standby NameNodes**. This avoids the NameNode becoming a single point of failure.

Cost Efficiency

A replication factor of 3 provides strong fault tolerance without exceeding the given limit. Large block sizes reduce metadata overhead, while compression and tiered storage reduce the amount of expensive storage required.

Justification

The design satisfies all constraints:

Constraint	Design Solution
Availability > 99.9%	HDFS replication + NameNode HA + rack awareness
Replication ≤ 3	Replication factor = 3
Cost efficient growth	Large blocks, compression, storage tiering
Large media files	Distributed DataNodes with scalable HDFS storage
Hardware failure	Automatic replica recovery

Thus, the proposed HDFS design provides high availability and fault tolerance while keeping storage expansion economically manageable.

2. MapReduce Workflow for Processing Logs Within 20 Minutes

Introduction

A media platform may generate millions of log records containing information about user activity, errors, page requests, and video access. The MapReduce workflow must complete processing within a **20-minute batch window** while using a limited number of cluster nodes.

Proposed Job Design

The workflow can be divided into the following stages:

HDFS Log Files → Input Splits → Mapper → Combiner → Shuffle and Sort → Reducer → HDFS Output

Mapper

The Mapper reads each log record and extracts useful information such as:

- IP address
- User ID
- Timestamp
- URL/video ID

- HTTP status
- Response time

For example:

video123, 200 → (video123, 1)

This indicates one successful access to the video.

Combiner

A Combiner performs local aggregation before data is transferred to reducers.

For example:

Instead of sending:

video123 → 1

video123 → 1

video123 → 1

the Combiner can send:

video123 → 3

This significantly reduces network traffic.

Reducer

Reducers receive all values belonging to the same key and calculate the final result.

For example:

video123 → [3, 5, 2]

The reducer produces:

video123 → 10 accesses

Task Distribution

Suppose the cluster has a limited number of DataNodes. The log files are divided into multiple input splits. Mapper tasks are executed close to the DataNodes containing the input data.

A suitable distribution is:

- Each large log file is divided into HDFS blocks.
- One Mapper is assigned approximately to each input split.
- Reducers are selected according to the number of unique keys and available cluster resources.
- Tasks are distributed across available nodes to prevent overloading a single node.
- Failed tasks are automatically re-executed by Hadoop.

Meeting the 20-Minute Constraint

The 20-minute requirement can be achieved by:

1. Using appropriate HDFS block sizes.
2. Running Mapper tasks in parallel.
3. Using a Combiner to reduce network traffic.
4. Using sufficient parallel Reducer tasks within the limited nodes.
5. Using data locality so that computation occurs close to the data.
6. Avoiding unnecessary MapReduce stages.
7. Monitoring slow or failed tasks and re-executing them.

Justification

Parallel execution allows multiple portions of the log data to be processed simultaneously. Combiners reduce shuffle traffic, while data locality reduces network overhead. Therefore, even with limited cluster nodes, the workflow can be designed to complete within the 20-minute batch window.

3. YARN Scheduler Policy for ETL, Reporting, and Machine Learning

Introduction

YARN manages cluster resources using components such as the ResourceManager, NodeManagers, ApplicationMasters, and containers. Since the cluster runs ETL, reporting, and machine learning jobs, the scheduler must balance fairness with priority.

Proposed Scheduler Policy

A **Capacity Scheduler** is suitable because it allows resources to be divided into queues and allows important workloads such as ETL to receive higher priority.

Queue Structure

A possible configuration is:

- **ETL Queue – 50% capacity**
- **Reporting Queue – 25% capacity**
- **Machine Learning Queue – 25% capacity**

ETL receives the highest priority because it is responsible for preparing data required by downstream applications.

Priority Policy

Workload	Priority	Resource Allocation
ETL	High	50%
Reporting	Medium	25%
Machine Learning	Medium	25%

Unused resources from one queue can be temporarily used by other queues, while minimum capacity is protected.

Fairness

Fairness is maintained by:

- Allocating minimum resources to every queue.
- Preventing one application from consuming the entire cluster.
- Sharing unused resources among workloads.
- Limiting resource usage for individual users or applications.

Node Failure Recovery

If a NodeManager fails:

1. YARN detects the node failure through heartbeat monitoring.
2. Containers running on the failed node are identified.
3. ApplicationMasters request replacement containers.
4. Failed tasks are restarted on healthy nodes.
5. Data is recovered from HDFS replicas.

Justification

The policy provides high priority to ETL without completely starving reporting and machine-learning workloads. Queue capacity ensures fairness, while YARN's task/container reallocation provides recovery from node failures.

4. Enterprise Data Ingestion from SAN, NAS, and Operational Databases

Introduction

An enterprise may have data distributed across SAN storage, NAS devices, and operational databases. The big data platform should ingest this data reliably while avoiding unnecessary duplication.

Proposed Architecture

The architecture can be designed as:

SAN/NAS/Databases → Apache NiFi/ETL → Validation & Deduplication → HDFS/Data Lake → Backup/DR Storage → Analytics

Components

1. Apache NiFi

Apache NiFi acts as the data ingestion and flow-management layer. It can receive data from different sources and route it to appropriate destinations.

2. Database Connectors

JDBC-based connectors can extract data from operational databases. Incremental extraction using timestamps or change tracking should be used instead of repeatedly copying complete tables.

3. SAN and NAS Ingestion

Files from SAN and NAS can be transferred through controlled ingestion flows. Only newly created or modified files should be transferred.

4. HDFS/Data Lake

HDFS provides distributed storage for the consolidated enterprise data. Data can be organized into zones such as:

- Raw zone
- Processed zone
- Curated zone

5. Deduplication

A unique record ID, file hash, timestamp, or source identifier can be used to identify duplicate data. Deduplication should occur before unnecessary copies are created in downstream storage.

6. Backup

Important datasets should be backed up to a separate storage system or disaster-recovery environment. HDFS replication provides operational fault tolerance, while an independent backup protects against larger failures or accidental deletion.

Justification

The architecture provides:

- **Reliable ingestion:** NiFi monitors and manages data flows.
- **Minimal duplication:** Incremental ingestion and hashing avoid unnecessary copies.
- **Backup reliability:** Independent backup/DR storage provides additional protection.
- **Centralized analytics:** Data from SAN, NAS, and databases becomes available in the big data platform.

- **Scalability:** HDFS can expand by adding DataNodes.

Therefore, the architecture combines controlled ingestion, deduplication, distributed storage, and independent backup.

5. End-to-End Hadoop Ecosystem Solution

Introduction

An enterprise Hadoop solution must securely move data, remain available during failures, and integrate with Apache NiFi or existing ETL pipelines.

Proposed Architecture

The complete architecture is:

Enterprise Sources → Apache NiFi/ETL → Secure Transfer → HDFS → YARN → Hive/Spark → Analytics & Reporting

1. Data Sources

Data can originate from:

- Operational databases
- SAN/NAS
- Application logs
- IoT devices
- External data sources

2. Apache NiFi / ETL Layer

NiFi manages data ingestion and routing. It can perform:

- Data collection
- Transformation
- Filtering
- Routing
- Monitoring
- Error handling

Existing ETL tools can also be integrated where required.

3. Secure Data Movement

Security should be provided using:

- **TLS/SSL** for data transmission.

- **Kerberos** for authentication within the Hadoop environment.
- **Encryption at rest** for sensitive HDFS data.
- **Access control policies** to restrict unauthorized users.
- Secure credentials and controlled service accounts.

4. Highly Available HDFS

HDFS should use:

- Active and Standby NameNodes
- Multiple DataNodes
- Replication factor of up to 3
- Rack-aware replica placement
- Automatic failure recovery

If one DataNode fails, another replica can provide the required data.

5. YARN Resource Management

YARN manages CPU and memory resources between different applications. Separate queues can be created for:

- ETL
- Reporting
- Machine Learning

Priority and capacity policies prevent one workload from consuming all resources.

6. Processing Layer

Hive can be used for SQL-based analysis and reporting, while **Spark** can be used for large-scale data processing and machine-learning workloads.

7. Monitoring and Recovery

The platform should continuously monitor:

- DataNode health
- NameNode health
- YARN resources
- NiFi data flows
- Failed jobs
- Storage utilization

Failed tasks can be automatically restarted, and HDFS replicas protect data when individual nodes fail.

Constraint Satisfaction

Constraint	Solution
Secure data movement	TLS/SSL, Kerberos, access controls
High availability	NameNode HA + DataNode replication
Data ingestion	Apache NiFi / ETL pipelines
Processing	YARN + Spark/Hive
Failure recovery	HDFS replication + YARN task recovery
Scalability	Add DataNodes as data grows
Data security	Authentication, authorization, encryption

Conclusion

The proposed end-to-end Hadoop ecosystem provides a secure and highly available platform for enterprise data processing. Apache NiFi or ETL pipelines provide controlled data ingestion, HDFS provides fault-tolerant storage, YARN manages resources, and Hive/Spark provide analytics and processing. The combination satisfies the requirements of **secure data movement, high availability, reliable processing, and integration with existing ETL workflows.**